

Faster Exact Counting of Labeled Chordal Graphs via Formal Power Series

Seonghyeon Park

Department of Computer Science and Engineering
Seoul National University
Seoul, Republic of Korea

Abstract

We give a counting-only acceleration of the dynamic program of Hébert-Johnson, Lokshtanov, and Vigoda for labeled chordal graphs. The existing analysis uses $O(n^7)$ arithmetic operations. We isolate its five-argument $\tilde{f}_p$ recurrence, retain the previously known regrouping by $r = x' + \ell'$, and expose an additional coefficient-extraction convolution. After an exponential-generating-function normalization in the remaining parameter, the self-reference becomes a first-order linear formal differential equation. Solving each (t, x, ℓ, z) block with truncated formal-power-series operations gives $O(n^5 M(n))$ field operations, where $M(d)$ is the cost of multiplying degree- d polynomials. A separate CRT corollary gives exact integer recovery from sufficiently many primes. We distinguish field-operation complexity from bit complexity and report independent recurrence, brute-force, modular, CRT, and implementation checks.

1 Introduction

Chordal graphs are graphs with no induced cycle of length at least four. Counting labeled chordal graphs is a natural test case for exact graph enumeration because perfect-elimination structure gives strong recursive descriptions while the labeled state space remains nontrivial. Hébert-Johnson, Lokshtanov, and Vigoda (HLV) gave the first polynomial-time exact counter for labeled chordal graphs and, more generally, for ω -colorable labeled chordal graphs, with an $O(n^7)$ arithmetic-operation analysis [3].

This paper improves the arithmetic-operation bound for the dominant recurrence. The contribution is not the HLV dynamic program, the regrouping $r = x' + \ell'$, generic polynomial multiplication, or generic formal-power-series (FPS) routines. The new step is to recognize that the regrouped recurrence contains both a polynomial coefficient-extraction convolution and a binomial-convolution self-reference. Under an exponential-generating-function normalization, the latter is a linear formal ODE and can be solved blockwise using standard FPS algorithms [1].

Our exact theorem is deliberately stated in two parts. The first concerns one execution over a suitable finite field. The second uses independent modular executions and CRT to recover the integer answer. We do not claim an unconditional improved bit or RAM bound, and we do not use the sampler from HLV.

2 The Counting Problem and the HLV Recurrence

Let $C(n, \omega)$ denote the number of labeled chordal graphs on vertex set $[n]$ with clique number at most ω . Since chordal graphs are perfect, this is equivalently the number of ω -colorable labeled chordal graphs. The unrestricted count is $C(n, n)$.

We give the exact state vocabulary needed here. For a clique exception set $X = [x]$, $\tilde{g}_1(t, x, k)$ counts graphs whose outside has exactly one component seeing all of X . The functions $\tilde{g}(t, x, k, z)$ and $\tilde{g}_p(t, x, k, z)$ impose, respectively, a neighbor in $X \setminus [z]$ for every outside component, or that no outside component sees all of X . The state $\tilde{f}_p(t, x, \ell, k, z)$ counts connected graphs with exception set $X = [x]$, leaf gate $L = [x+1, x+\ell]$, $X \cup L$ a clique, all components outside it evaporating at time $t-1$, none seeing all of $X \cup L$, and each having a neighbor in $(X \cup L) \setminus [z]$. Its domain is $t \geq 2$, $x \geq 0$, $\ell \geq 1$, $z \leq x$, and $k \geq 0$.

The remaining HLV states are $g, g_1, g_{\geq 2}, f, \tilde{f}$ and their boundary variants, distinguished only by evaporation time, connected-component count, and whether components see all of the boundary. The source extracts connected counts as $c(n) = \sum_t \tilde{g}_1(t, 0, n)$ and all counts as $a(n) = \sum_{k=1}^n \binom{n-1}{k-1} c(k) a(n-k)$ with $a(0) = 1$.

The base cases used by the source are $g(0, x, 0, z) = 1$ and $g(0, x, k, z) = 0$ for $k > 0$; $\tilde{g}(t, x, 0, z) = \tilde{g}_p(t, x, 0, z) = 1$; $\tilde{g}_1$ and $\tilde{g}_{\geq 2}$ vanish when $t = 0$ or $k = 0$; f vanishes when $x + \ell > \omega$, $t = 0$, or (for $t \geq 2$) $k = 0$, while $f(1, x, \ell, k)$ is one exactly when $k = 0$ and $x + \ell \leq \omega$; and $\tilde{f}_p$ vanishes when $t = 1$ or $k = 0$. These conditions define the zero entries used in the block induction.

For the state being accelerated, the exact source recurrence is

$$F_k = \sum_{q=1}^k \binom{k-1}{q-1} \sum_{u=0}^x \sum_{i=0}^{\ell} \binom{\ell}{i} G_{u+i, q} W(u, i) T_i(k-q),$$

where $F_k = \tilde{f}_p(t, x, \ell, k, z)$, $G_{r, q} = \tilde{g}_1(t-1, r, q)$, the prime means $0 < u+i < x+\ell$, and

$$W(u, i) = \begin{cases} \binom{x}{u}, & i > 0, \\ \binom{x}{u} - \binom{z}{u}, & i = 0, \end{cases} \quad T_i(j) = \begin{cases} \tilde{f}_p(t, x+i, \ell-i, j, z), & i < \ell, \\ \tilde{g}_p(t-1, x+\ell, j, z), & i = \ell. \end{cases}$$

Thus $i = 0$ is the self-reference, $0 < i < \ell$ strictly reduces ℓ , and $i = \ell$ is a boundary call. The proper-subset condition gives $r = u+i \leq x+\ell-1$.

3 The Additional Convolution

Fix (t, x, ℓ, z) and put $K = n - x - \ell$. For $j \geq 0$, write $F_j = \tilde{f}_p(t, x, \ell, j, z)$. After the known regrouping by $r = u + i$, the recurrence has the form

$$F_k = \sum_{q=1}^k \binom{k-1}{q-1} \sum_{r=1}^{x+\ell-1} G_{r,q} H_{r,k-q},$$

where $G_{r,q} = \tilde{g}_1(t-1, r, q)$ and $H_{r,j}$ denotes the inner (u, i) sum only as an algebraic abbreviation. The implementation does not materialize a dense two-index table H : it computes the known forcing directly as a sum of univariate products below. Splitting off $i = 0$ gives

$$H_{r,j} = a_r F_j + b_r(j), \quad a_r = \binom{x}{r} - \binom{z}{r}.$$

The remaining polynomial is

$$D_j(U) = \sum_{i=1}^{\ell-1} \binom{\ell}{i} F_j^{(i)} U^i + G_j U^\ell,$$

where $F_j^{(i)}$ is the already computed smaller- ℓ state and G_j is the endpoint boundary value. Then $b_r(j)$ is the coefficient of U^r in $D_j(U)(1+U)^x$. Explicitly, for fixed r , the coefficient receives the terms with $i \in [\max(1, r-x), \min(r, \ell-1)]$, weighted by $\binom{\ell}{i} \binom{x}{r-i}$, together with the endpoint term when $r \geq \ell$, weighted by $G_j \binom{x}{r-\ell}$. These are exactly the cases $u = r - i$ in the source sum; the excluded $i = 0$ case is the self-reference, and the proper-subset condition removes $r = x + \ell$. Thus the $a_r F_j$ terms form a binomial convolution in q and j , while all $b_r(j)$ terms are known forcing terms within the block.

4 EGF and Formal ODE

Define

$$R_r(Y) = \sum_{q=1}^K G_{r,q} \frac{Y^{q-1}}{(q-1)!}, \quad B_r(Y) = \sum_{j=0}^K b_r(j) \frac{Y^j}{j!},$$

and form the forcing series directly as

$$Q(Y) = \sum_{r=1}^{s-1} R_r(Y) B_r(Y).$$

This is a sum of $s - 1$ univariate products; it is not a dense two-dimensional matrix contraction. Also put $A_q = \sum_r a_r G_{r,q}$ and define

$$P(Y) = \sum_{q=1}^K A_q \frac{Y^{q-1}}{(q-1)!}.$$

72 With $F(Y) = \sum_{k=1}^K F_k Y^k / k!$ and $F_0 = 0$, coefficient extraction yields

$$F'(Y) = P(Y)F(Y) + Q(Y), \quad F(0) = 0.$$

73 The unique formal solution is

$$F(Y) = E(Y) \int_0^Y E(V)^{-1} Q(V) dV, \quad E(Y) = \exp\left(\int_0^Y P(V) dV\right).$$

74 All series are truncated at degree K . The strict decrease of ℓ in the interior terms
 75 makes the block evaluation acyclic in ascending ℓ ; the endpoint is a previously
 76 available $\tilde{g}_p$ value. Formal uniqueness follows coefficient by coefficient: once
 77 $F_0, \dots, F_m$ are known, the coefficient of Y^m on the right determines F_{m+1} ,
 78 because P has no constant-term contribution from an unknown F_{m+1} and Q is
 79 already known. Equivalently, the integrating factor has constant term 1 and is
 80 invertible in the truncated power-series ring.

81 For the coefficient check, the coefficient of Y^{k-1} in $P(Y)F(Y)$ is

$$\sum_{q=1}^k \frac{A_q}{(q-1)!} \frac{F_{k-q}}{(k-q)!}.$$

82 Multiplication by $(k-1)!$ gives $\sum_q \binom{k-1}{q-1} A_q F_{k-q}$, exactly the homogeneous part
 83 of the recurrence. Similarly, the coefficient of Y^{k-1} in $R_r B_r$ gives $G_{r,q} b_r (k-q)/((q-1)!(k-q)!)$, and multiplication by $(k-1)!$ gives the forcing term. This
 84 proves why $(q-1)!$, rather than $q!$, is required.
 85

86 5 Complexity

87 For a fixed block, the $K+1$ products $D_j(U)(1+U)^x$ cost $O(KM(s))$. Forming
 88 P costs $O(sK)$ scalar operations, while $Q = \sum_r R_r B_r$ costs $O(sM(K))$. The
 89 integrating-factor inverse, exponential, integrations, and multiplications cost
 90 $O(M(K))$ up to logarithmic factors. Hence the block costs $O(KM(s) + sM(K))$.
 91 The block returns the entire vector $F_0, \dots, F_K$ in one solve.

92 For fixed s , there are $O(s^2)$ choices of (x, ℓ, z) after summing over $z \leq x$,
 93 and $O(n)$ choices of t . Since $K = n - s$, summing the block cost gives

$$\sum_{s=1}^n O(ns^2((n-s)M(s) + sM(n-s))) = O(n^5 M(n))$$

94 when M is monotone up to constant factors and $M(d) = \Omega(d)$. The remaining
 95 HLV families contribute, under their memoized valid domains, $O(n^6)$ for the
 96 g -family, $O(n^4)$ for the one-/multi-component boundary families, $O(n^5)$ for the
 97 remaining f terms, and at most $O(n^5)$ for repeated connected-count extraction
 98 in the all-graph recurrence. Each is absorbed by the displayed block sum. A
 99 block is keyed by the active (n, p) scope together with (t, x, z, ℓ, ω) ; its vector is
 100 created at most once and all later k requests are table lookups.

Theorem 1 (Modular counting). *For $1 \leq \omega \leq n$ and a finite field of characteristic greater than n , one execution of the abstract blockwise algorithm computes $C(n, \omega)$ using $O(n^5 M_p(n))$ field operations, where $M_p(d)$ dominates degree- d multiplication and every truncated-FPS primitive used by the integrating-factor solver, including logarithmic overhead. The supplied NTT implementation realizes this theorem for admissible NTT primes and transform lengths; it rejects unsupported lengths rather than silently applying an invalid transform.*

Proof. The recurrence identities are integer identities. Since $p > n$, every factorial denominator through the truncation degree is invertible, so the EGF normalization, ODE solution, and coefficient extraction are valid in $\mathbb{F}_p$. The block dependency order proves correctness by induction on ℓ and the outer HLV order. The preceding sum charges the operations. $\square$

6 Correctness and Evaluation Order

Fix (n, ω, p) for one modular execution. At time t , evaluate the $\tilde{f}_p$ blocks in increasing ℓ , then the remaining $\tilde{f}$, f , $\tilde{g}_1$, $\tilde{g}_{\geq 2}$, boundary, g , and connected/all-graph families. Zero and boundary cases are returned before recursive calls. Every interior $\tilde{f}_p$ edge has target leaf parameter $\ell - i < \ell$; its $i = \ell$ edge and all other boundary edges decrease t . The remaining same-layer recurrences decrease k in their recursive argument, and the outer all-graph recurrence decreases the vertex budget. The complete family-by-family edge list and domains are given in the accompanying dependency proof.

The only same-state reference is the $i = 0$ term of $\tilde{f}_p$. It is solved by the formal ODE, not by recursion, and the coefficient recurrence determines F_{m+1} from coefficients through F_m . Induction over this block order and then the outer HLV order proves that the block solver preserves every source recurrence value. The cache is cleared for each fixed (n, p, ω) execution; within that scope the key (t, x, z, ℓ, ω) determines $K = n - x - \ell$ and stores the full vector $F_0, \dots, F_K$. Later requests for another k are lookups only.

7 Exact Integer Recovery

There are at most $2^{\binom{n}{2}}$ simple labeled graphs, hence

$$0 \leq C(n, \omega) \leq B := 2^{\binom{n}{2}}.$$

Choose pairwise-coprime admissible primes $p_i > n$ whose product P exceeds B . Run the modular theorem for every p_i and reconstruct the unique residue in $[0, P)$ by canonical nonnegative CRT. Since the desired answer lies in $[0, B]$ and $P > B$, it is the unique nonnegative integer in the allowed interval; the implementation uses this one-sided reconstruction. Intermediate DP values need not be reconstructed: modular reduction of the integer recurrence computes the reduction of the final integer value.

138 **Corollary 2** (Exact counting). *The modular theorem together with sufficiently*
139 *many primes $p_i > n$ gives exact integer recovery of $C(n, \omega)$ by CRT.*

140 This corollary is an exactness statement, not a claim that CRT has the cost
141 of one modular execution. The abstract theorem allows any suitable fast mul-
142 tiplication backend; the supplied reference implementation uses NTT-friendly
143 primes and validates each required transform length. We leave the $\tilde{O}(n^8)$ bit-
144 cost route conditional on the detailed arithmetic assumptions described in the
145 supplement.

146 8 Implementation and Validation

147 We implemented three independent layers: a literal reference recurrence, a
148 transformed-naive recurrence, and a modular transformed-fast recurrence with
149 NTT/FPS primitives. The test suite includes randomized identity checks, ODE
150 coefficient checks, block comparisons, CRT reconstruction, mutation checks, a
151 block-cache regression, and brute-force chordality enumeration.

152 The verification package provides the literal and transformed implemen-
153 tations used for the checks below. Our independent suite contains 34 tests,
154 and the separate mutation suite contains 10 tests. The unrestricted values for
155 $n = 1, \dots, 9$ begin

1, 2, 8, 61, 822, 18154, 617675, 30888596, 2192816760,

156 and agree with the published sequence A058862 [5]. Brute-force simplicial-
157 elimination checks agree through six vertices. An instrumented $n = 8, \omega = 8$
158 run created 406 distinct blocks and served 722 later queries from cached block
159 vectors.

160 The canonical entry point is `python scripts/verify_all.py`; it runs both
161 test suites, Python compilation, and certificate generation. The certificate
162 records the block key, creation count, hit count, and truncation parameters.
163 The auxiliary recurrence ledger and the theorem above specify the field-level
164 assumptions under which the FPS replacement is valid.

165 9 Related Work and Limitations

166 The HLV paper is the source of the recurrence and the $O(n^7)$ arithmetic bound
167 [3]. The 2025 dissertation records the $r = x' + \ell'$ regrouping and six-argument
168 helper, so those steps are prior art [2]. The present contribution begins with the
169 additional coefficient-extraction and EGF/ODE reduction. Sun’s 2026 work
170 uses the labeled HLV recurrence as a base case for an equivariant/unlabeled
171 divisor-bundle computation; its available paper and README do not state this
172 acceleration [4].

173 Our theorem is not an unconditional bit-complexity improvement. The exact
174 CRT route is rigorous, but the cost of arbitrary-field multiplication, prime gen-

175 eration, and every auxiliary table is not reduced here to a fully implementation-
 176 level RAM theorem. Priority is stated conservatively: to the best of our knowl-
 177 edge, no inspected source states the additional EGF/FPS acceleration, but the
 178 unavailable source history and lack of author responses prevent an absolute
 179 priority claim.

180 10 Auxiliary Recurrence Ledger

181 For reference, the auxiliary calls used by the top-down HLV program are listed
 182 here; this makes the global cost claim checkable without consulting implemen-
 183 tation code. With binomial coefficients interpreted as zero outside their natural
 184 ranges,

$$\begin{aligned}
 \tilde{g}_1(t, x, k) &= \sum_{\ell=1}^k \binom{k}{\ell} f(t, x, \ell, k - \ell), \\
 f(t, x, \ell, k) &= \sum_{q=1}^k \binom{k}{q} \tilde{f}(t, x, \ell, q) g(t - 2, x + \ell, k - q, x), \\
 g(t, x, k, z) &= \sum_{q=0}^k \binom{k}{q} \tilde{g}(t, x, q, z) g(t - 1, x, k - q, z), \\
 \tilde{g}(t, x, k, z) &= \sum_{q=1}^k \sum_{u=1}^x \left(\binom{x}{u} - \binom{z}{u} \right) \binom{k-1}{q-1} \tilde{g}_1(t, u, q) \tilde{g}(t, x, k - q, z), \\
 \tilde{f}(t, x, \ell, k) &= \tilde{f}_p(t, x, \ell, k) + \sum_{q=1}^k \binom{k}{q} \tilde{g}_1(t - 1, x + \ell, q) \tilde{f}_p(t, x, \ell, k - q) \\
 &\quad + \sum_{q=1}^k \binom{k}{q} \tilde{g}_{\geq 2}(t - 1, x + \ell, q) \tilde{g}_p(t - 1, x + \ell, k - q, x), \\
 \tilde{g}_{\geq 2}(t, x, k) &= \sum_{q=1}^{k-1} \binom{k-1}{q-1} \tilde{g}_1(t, x, q) (\tilde{g}_1(t, x, k - q) + \tilde{g}_{\geq 2}(t, x, k - q)), \\
 \tilde{g}_p(t, x, k, z) &= \sum_{q=1}^k \sum_{u=1}^{x-1} \left(\binom{x}{u} - \binom{z}{u} \right) \binom{k-1}{q-1} \tilde{g}_1(t, u, q) \tilde{g}_p(t, x, k - q, z),
 \end{aligned}$$

192 and $\tilde{f}_p(t, x, \ell, k) = \tilde{f}_p(t, x, \ell, k, x)$. These recurrences, together with the base
 193 cases above, define the complete arithmetic program. The displayed $\tilde{f}_p$ identity
 194 in Section 3 is the only one replaced by the FPS block solver.

195 The block implementation first materializes the lower- ℓ vectors, boundary
 196 vector, and all $G_{r,q}$ values; it then forms the $D_j(1 + U)^x$ products, the $R_r B_r$
 197 products, and the integrating-factor solution. A cache lookup keyed by the
 198 active (n, p) scope together with (t, x, z, ℓ, ω) returns the stored vector entry for
 199 any subsequent k . Thus no caller can re-run polynomial work for a different k .

200 11 Conclusion

201 The HLV labeled chordal-graph counter has a block structure that supports a
202 further algebraic acceleration. The resulting exact counting theorem separates
203 modular field operations from CRT recovery and provides a clean counting-only
204 response to the faster-exact-counting direction. The main remaining opportuni-
205 ties are a complete implementation-level bit analysis and broader independent
206 priority confirmation.

207 References

- 208 [1] Richard P. Brent and H. T. Kung. Fast algorithms for manipulating formal
209 power series. *Journal of the ACM*, 25(4):581–595, 1978.
- 210 [2] Ursula Hébert-Johnson. *Efficient Algorithms for Graph Counting and Sam-*
211 *pling from a Parameterized Perspective*. PhD thesis, University of California,
212 Santa Barbara, 2025.
- 213 [3] Ursula Hébert-Johnson, Daniel Lokshtanov, and Eric Vigoda. Counting and
214 sampling labeled chordal graphs in polynomial time. *Leibniz International*
215 *Proceedings in Informatics*, 274:58:1–58:19, 2023. arXiv:2308.09703.
- 216 [4] Matthew Sun. Counting unlabeled chordal graphs by equivariant evapora-
217 tion. *arXiv preprint arXiv:2607.02613*, 2026.
- 218 [5] Matthew Sun et al. Number of chordal labeled graphs (connected or not)
219 with n nodes. OEIS A058862, 2026. Accessed 2026-09-03.